

Foundations of Artificial Intelligence

Part VI: Introduction to Neural Networks

Department of Computer Science

Lecture Roadmap

1. Biological Inspiration
2. The Perceptron
3. Multilayer Neural Networks
4. Backpropagation

Learning Objectives

By the end of this lecture, you should be able to:

- Describe the biological inspiration behind artificial neural networks.
- Define a perceptron and apply the perceptron learning rule.
- Distinguish single-layer from multilayer networks and explain why hidden layers matter.
- Explain the forward pass, loss functions, and training loop conceptually.
- Articulate what backpropagation does and how the chain rule makes it efficient.

Biological Inspiration

Why Neural Networks?

Neural networks represent a shift from symbolic AI to learning from data.

Key motivations:

- Brain-inspired: learning through example, adaptability.
- Universal approximators: can learn any function (given enough capacity).
- Powerful for high-dimensional data (images, audio, text).
- Foundation for deep learning and modern AI.

Conceptual goal: Understand how neurons work together to learn patterns.

The Biological Neuron

Structure:

- **Dendrites:** Receive signals from other neurons.
- **Cell body (soma):** Integrates incoming signals.
- **Axon:** Transmits signal to other neurons.
- **Synapses:** Junctions where signals pass; strength varies through learning.

Key insight: Learning means adjusting synaptic weights through experience.

Artificial simplification:

- Inputs $\leftrightarrow$ dendrites
- Weights $\leftrightarrow$ synaptic strengths
- Activation function $\leftrightarrow$ firing threshold
- Output $\leftrightarrow$ axon signal

From Neurons to Networks

A single neuron is too simple to be useful alone.

Scale in biology:

- Human brain: $\sim$ 86 billion neurons.
- Each neuron connects to thousands of others.
- Intelligence emerges from massive, structured connectivity.

Scale in artificial networks:

- Layers of artificial neurons organised in parallel.
- Early networks: tens of neurons.
- Modern LLMs: billions of parameters.

The architecture and learning algorithm matter as much as size.

The Perceptron

The Perceptron

Simplest artificial neuron (Rosenblatt, 1958):

$$\text{output} = \begin{cases} 1 & \text{if } \sum_i w_i x_i + b \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

Components:

- **Inputs** $x_1, \dots, x_n$: feature values.
- **Weights** $w_1, \dots, w_n$: importance of each feature (learned).
- **Bias** b : shifts the decision threshold.
- **Activation**: step function (outputs 0 or 1).

Geometric view: The perceptron defines a *hyperplane* that separates two classes.

Perceptron Learning Rule

How does a perceptron learn?

Algorithm:

1. Initialise weights w_i and bias b to small random values.
2. For each training example (x, y) :
 - Compute prediction: $\hat{y} = \text{step}(\sum_i w_i x_i + b)$
 - If $\hat{y} \neq y$ (wrong):

$$w_i \leftarrow w_i + \alpha (y - \hat{y}) x_i, \quad b \leftarrow b + \alpha (y - \hat{y})$$

where $\alpha > 0$ is the *learning rate*.

3. Repeat until convergence (or max epochs).

Key idea: Nudge weights toward correct classification on each mistake.

Perceptron Limitations: The XOR Problem

Fatal limitation: Perceptrons can only learn **linearly separable** patterns.

XOR truth table:

x_1	x_2	$x_1 \oplus x_2$
0	0	0
0	1	1
1	0	1
1	1	0

No single straight line can separate the two classes.

Impact (1969): Minsky & Papert's analysis caused the first "AI winter" for neural nets.

Solution: Stack multiple perceptrons into a *multilayer network*.

Multilayer Neural Networks

Multilayer Network Architecture

Three types of layers:

- **Input layer:** Raw feature values (no computation).
- **Hidden layer(s):** Learn intermediate representations.
- **Output layer:** Produces final prediction.

Why hidden layers?

- Stacking linear functions is still linear — no gain.
- Non-linear activation functions break linearity at each layer.
- Hidden layers learn progressively abstract features.

Depth vs. width:

- *Deep* network: many layers (complex hierarchical features).
- *Wide* network: many units per layer (more capacity per layer).

Activation Functions

Purpose: Introduce non-linearity between layers.

Common choices:

- **Step:** $1[z \geq 0]$ — original perceptron; not differentiable.
- **Sigmoid:** $\sigma(z) = \frac{1}{1 + e^{-z}}$ — smooth, range $(0, 1)$.
- **Tanh:** $\tanh(z)$ — symmetric around 0, range $(-1, 1)$.
- **ReLU:** $\max(0, z)$ — simple, fast; default in modern networks.

Why differentiability matters: Gradient-based training requires computing $\frac{d}{dz}[\text{activation}]$.

ReLU's simplicity and avoidance of vanishing gradients make it the go-to choice.

Forward Pass

Goal: Compute the network's output given an input.

For layer l :

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}, \quad a^{(l)} = \sigma(z^{(l)})$$

where $W^{(l)}, b^{(l)}$ are the weight matrix and bias, $a^{(l)}$ is the layer output.

Loss function: Measures how wrong the prediction is.

- **Regression** (MSE): $L = \frac{1}{m} \sum_i (y_i - \hat{y}_i)^2$
- **Classification** (cross-entropy):
$$L = -\frac{1}{m} \sum_i [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)]$$

Goal of training: Minimise L over all training examples.

Backpropagation

The Training Problem

Challenge: We have weights across many layers. How do we know which weights to change and by how much?

Gradient descent updates weights in the direction that reduces loss:

$$w \leftarrow w - \alpha \frac{\partial L}{\partial w}$$

where α is the *learning rate*.

Problem: Computing $\frac{\partial L}{\partial w}$ for every weight in a deep network is expensive if done naively.

Solution: Backpropagation — efficiently compute all gradients in a single backward pass using the chain rule.

Backpropagation: Conceptual Overview

Four steps:

1. **Forward pass:** Compute predictions and loss L .
2. **Output gradient:** Compute $\frac{\partial L}{\partial \hat{y}}$ at the output layer.
3. **Backward pass:** Propagate error signals layer-by-layer *backward* through the network.
4. **Weight update:** Adjust every weight $w \leftarrow w - \alpha \frac{\partial L}{\partial w}$.

Key property: Each layer only needs the gradient from the layer *above* it — no redundant computation.

Backprop makes training deep networks computationally feasible.

Backpropagation: The Chain Rule

Core mechanism: The chain rule of calculus.

Example — gradient w.r.t. weights in layer 1 of a 2-layer network:

$$\frac{\partial L}{\partial W^{(1)}} = \underbrace{\frac{\partial L}{\partial \hat{y}}}_{\text{output error}} \cdot \underbrace{\frac{\partial \hat{y}}{\partial z^{(2)}}}_{\sigma'} \cdot \underbrace{\frac{\partial z^{(2)}}{\partial a^{(1)}}}_{W^{(2)}} \cdot \underbrace{\frac{\partial a^{(1)}}{\partial z^{(1)}}}_{\sigma'} \cdot \underbrace{\frac{\partial z^{(1)}}{\partial W^{(1)}}}_{a^{(0)}}$$

Intuition:

- Start with the error at the output.
- Multiply through each layer's local gradient.
- Each layer “receives” how much it contributed to the error.

Training Loop

Standard training procedure:

1. **Initialise** weights randomly (small values).
2. For each *epoch* (pass over training data):
 - 2.1 **Forward**: Compute $\hat{y}$ and loss L .
 - 2.2 **Backward**: Compute all gradients via backprop.
 - 2.3 **Update**: $W \leftarrow W - \alpha \nabla_W L$.
3. Monitor loss on a *validation set*; stop when it no longer improves.

Key hyperparameters:

- Learning rate α : too large $\rightarrow$ diverge; too small $\rightarrow$ slow.
- Number of layers and units per layer.
- Batch size (how many examples per update).
- Regularisation strength.

Overfitting and Generalisation

Overfitting: The model memorises training data and performs poorly on new data.

Signs: Training loss decreases, but validation loss increases.

Common remedies:

- **More data:** Usually the most effective fix.
- **Dropout:** Randomly zero out units during training (forces redundancy).
- **L2 regularisation:** Penalise large weights: $L_{\text{reg}} = L + \lambda \|W\|^2$.
- **Early stopping:** Stop training when validation loss stops improving.
- **Batch normalisation:** Normalise layer inputs; stabilises training.

Strengths and Challenges

Strengths:

- **Universal approximation:** A network with one hidden layer can approximate any continuous function (given enough units).
- Learn complex non-linear patterns automatically.
- End-to-end learning: raw input to prediction without hand-crafted features.

Challenges:

- **Local minima & saddle points:** Gradient descent is not guaranteed to find the global optimum.
- **Vanishing/exploding gradients:** Gradients can shrink or grow exponentially in deep nets.
- **Interpretability:** Hard to explain individual decisions (black box).
- **Data and compute:** Large networks need large datasets and significant compute.

Summary

What we covered:

- Biological inspiration: neurons, synapses, and learning as weight adjustment.
- Perceptron: linear classifier, learning rule, XOR limitation.
- Multilayer networks: hidden layers, activation functions, forward pass.
- Backpropagation: chain rule, efficient gradient computation, training loop.
- Practical concerns: overfitting, regularisation, hyperparameter tuning.

Key takeaway: Neural networks learn by adjusting millions of weights to minimise a loss function — backpropagation makes this tractable.

Further study:

- Convolutional Neural Networks (CNNs) for images.
- Recurrent Neural Networks (RNNs) for sequences.